Projekt TUC

Wygenerowano przez Doxygen 1.10.0

1 Indeks klas 1
1.1 Lista klas . . . 1

2 Indeks plików 3
2.1 Lista plików . . . 3

3 Dokumentacja klas 5
3.1 Dokumentacja struktury GATE . . . 5
3.1.1 Opis szczegółowy . . . 5
3.1.2 Dokumentacja atrybutów składowych . . . 5
3.1.2.1 inputNodes . . . 5
3.1.2.2 Name . . . 5
3.1.2.3 out . . . 6
3.2 Dokumentacja struktury PARAM . . . 6
3.2.1 Opis szczegółowy . . . 6
3.2.2 Dokumentacja atrybutów składowych . . . 6
3.2.2.1 i . . . 6
3.2.2.2 o . . . 6
3.2.2.3 u . . . 6

4 Dokumentacja plików 7
4.1 Dokumentacja pliku DeklaracjaFunkcji.h . . . 7
4.1.1 Opis szczegółowy . . . 8
4.1.2 Dokumentacja funkcji . . . 8
4.1.2.1 CheckParameters() . . . 8
4.1.2.2 EvaluateOut() . . . 8
4.1.2.3 ReadCircuit() . . . 9
4.1.2.4 ReadInputsAndWrite() . . . 10
4.1.2.5 ReadParameters() . . . 10
4.1.2.6 TranslateStringToEnum() . . . 11
4.2 DeklaracjaFunkcji.h . . . 11
4.3 Dokumentacja pliku Funkcje.cpp . . . 12
4.3.1 Opis szczegółowy . . . 12
4.3.2 Dokumentacja funkcji . . . 13
4.3.2.1 CheckParameters() . . . 13
4.3.2.2 EvaluateOut() . . . 13
4.3.2.3 ReadCircuit() . . . 14
4.3.2.4 ReadInputsAndWrite() . . . 14
4.3.2.5 ReadParameters() . . . 15
4.3.2.6 TranslateStringToEnum() . . . 15
4.4 Dokumentacja pliku MainKod.cpp . . . 16
4.4.1 Opis szczegółowy . . . 17
4.5 Dokumentacja pliku TypyiStructy.h . . . 17

4.5.1 Opis szczegółowy 18
4.6 TypyiStructy.h 18

Rozdział 1

Indeks klas

1.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

GATE
Struktura odzwierciedlajaca bramki 5
PARAM
Struktura zawierajaca w sobie argumenty 6

Rozdział 2

Indeks plików

2.1 Lista plików

Tutaj znajduje się lista wszystkich udokumentowanych plików z ich krótkimi opisami:

DeklaracjaFunkcji.h
Deklaracja funkcji zawartych w programie 7
Funkcje.cpp
Funkcje potrzebne do programu 12
MainKod.cpp
Program tworzacy uklady cyfrowe i liczacy wartosci 16
TypyiStructy.h
Header zawierajacy struktury i typy wyliczeniowe 17

Rozdział 3

Dokumentacja klas

3.1 Dokumentacja struktury GATE

Struktura odzwierciedlajaca bramki.

```
#include <TypyiStructy.h>
```

Atrybuty publiczne

- GateType Name
- std::vector< int > inputNodes
- GateValue out = GateValue::UNIND

3.1.1 Opis szczegółowy

Struktura odzwierciedlajaca bramki.

3.1.2 Dokumentacja atrybutów składowych

3.1.2.1 inputNodes

```
std::vector<int> GATE::inputNodes
```

Numery wezlow wchodzacych do danej bramki

3.1.2.2 Name

```
GateType GATE::Name
```

Typ bramki

3.1.2.3 out

```
GateValue GATE::out = GateValue::UNIND
```

Aktualna wartosc na wyjsciu bramki

Dokumentacja dla tej struktury została wygenerowana z pliku:

- TypyiStructy.h

3.2 Dokumentacja struktury PARAM

Struktura zawierajaca w sobie argumenty.

```
#include <TypyiStructy.h>
```

Atrybuty publiczne

- std::string u
- std::string i
- std::string o

3.2.1 Opis szczegółowy

Struktura zawierajaca w sobie argumenty.

3.2.2 Dokumentacja atrybutów składowych

3.2.2.1 i

```
std::string PARAM::i
```

Plik wejscia z wartosciami na wejsciach

3.2.2.2 o

```
std::string PARAM::o
```

Plik wyjsciowy z wartosciami na wejsciach i wyjsciach

3.2.2.3 u

```
std::string PARAM::u
```

Plik wejscia z obwodem

Dokumentacja dla tej struktury została wygenerowana z pliku:

- TypyiStructy.h

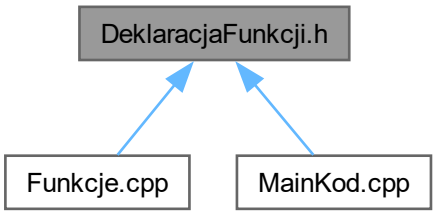
Rozdział 4

Dokumentacja plików

4.1 Dokumentacja pliku DeklaracjaFunkcji.h

Deklaracja funkcji zawartych w programie.

Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Funkcje

- void CheckParameters (int argc, char ∗argv[ ])

  Funkcja sprawdzajaca czy zgadza sie liczba podanych argumentow.
- void ReadParameters (int argc, char ∗argv[ ], struct PARAM &parameters)

  Funkcja czytajaca argumenty i podajaca je do struktury z parametrami.
- void ReadCircuit (std::string input, std::vector< struct GATE > &VectorGates, std::vector< int > &OUTPUTS, std::vector< int > &INPUTS)

  Funkcja czytajaca uklad z pliku i budujaca go.
- void ReadInputsAndWrite (std::string input2, std::string output, std::vector< struct GATE > &VectorGates, std::vector< int > OUTPUTS, std::vector< int > INPUTS)

  Funkcja czytajaca wartosci na wejsciach z pliku i wprowadzajaca wartosci wejsc do ukladu oraz wprowadzajaca wartosci do kazdej bramki i wypisujaca wartosci wyjsc.
- enum GateType TranslateStringToEnum (std::string original)

  Funkcja "tlumaczaca" stringi z pliku na enum.
- void EvaluateOut (int nodeID, std::vector< struct GATE > &VectorGates)

  Funkcja wyliczajaca wartosc wyjscia z wartosci znajdujacej sie na wejsciu bramki.

4.1.1 Opis szczegółowy

Deklaracja funkcji zawartych w programie.

Autor

Igor Szyk

Wersja

0.1

Data

2024-01-28

4.1.2 Dokumentacja funkcji

4.1.2.1 CheckParameters()

```
void CheckParameters (
            int argc,
            char * argv[] )
```

Funkcja sprawdzajaca czy zgadza sie liczba podanych argumentow.

Parametry

argc	parametr dla ilosci argumentow
argv[]	parametr tablicy dla nazw plikow uzywanych w programie

Oto graf wywoływań tej funkcji:

4.1.2.2 EvaluateOut()

```
void EvaluateOut (
            int nodeID,
            std::vector< struct GATE > & VectorGates )
```

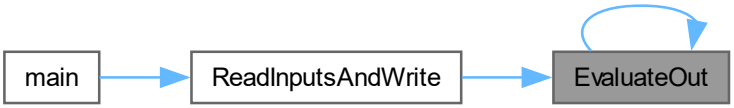
Funkcja wyliczajaca wartosc wyjscia z wartosci znajdujacej sie na wejsciu bramki.

Parametry

nodeID	numer wezla z ktorego aktualnie bedzie wyznaczana wartosc na wyjsciu
VectorGates	wektor zawierajacy uklad, na ktorym bedzie sie operowac

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:



4.1.2.3 ReadCircuit()

```
void ReadCircuit (
            std::string input,
            std::vector< struct GATE > & VectorGates,
            std::vector< int > & OUTPUTS,
            std::vector< int > & INPUTS )
```

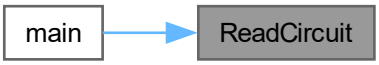
Funkcja czytajaca uklad z pliku i budujaca go.

Parametry

input	plik z ktorego czyta sie budowe ukladu
VectorGates	wektor do ktorego wpisuje sie wyczytany uklad z pliku
OUTPUTS	wektor do ktorego wypisuje sie numery bramek wyjscia
INPUTS	wektor do ktorego wypisuje sie numery bramek wejscia

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:



4.1.2.4 ReadInputsAndWrite()

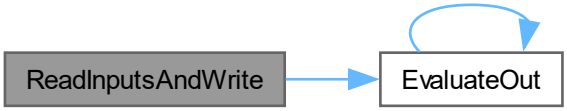
```
void ReadInputsAndWrite (
            std::string input2,
            std::string output,
            std::vector< struct GATE > & VectorGates,
            std::vector< int > OUTPUTS,
            std::vector< int > INPUTS )
```

Funkcja czytajaca wartosci na wejsciach z pliku i wprowadzajaca wartosci wejsc do ukladu oraz wprowadzajaca wartosci do kazdej bramki i wypisujaca wartosci wyjsc.

Parametry

input2	plik z ktorego czyta sie wartosci na wejsciach
output	plik do ktorego wypisuje sie wartosci wejsc i wyjsc
VectorGates	wektor na ktorym czyta sie wartosci wyjsc potrzebnych bramek
OUTPUTS	wektor numerow wyjsc do ktorych szuka sie wartosci
INPUTS	wektor numerow wejsc, ktore wykorzystuje sie do sprawdzenia czy zgadza sie ilosc wartosci z iloscia numerow wezlow

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

4.1.2.5 ReadParameters()

```
void ReadParameters (
            int argc,
            char * argv[],
            struct PARAM & parameters )
```

Funkcja czytajaca argumenty i podajaca je do struktury z parametrami.

Parametry

argc	parametr dla ilosci argumentow
argv[]	parametr tablicy dla nazw plikow uzywanych w programie
parameters	parametr struktury do ktorej wstawia sie wyczytane argumenty

Oto graf wywoływań tej funkcji:

4.1.2.6 TranslateStringToEnum()

```
enum GateType TranslateStringToEnum (
            std::string original )  [strong]
```

Funkcja "tlumaczaca" stringi z pliku na enum.

Parametry

original	parametr zawierajacy oryginalny tekst w stringu

Zwraca

typ bramki w enum

Oto graf wywoływań tej funkcji:

4.2 DeklaracjaFunkcji.h

Idź do dokumentacji tego pliku.

```
#pragma once


void CheckParameters(int argc, char* argv[]);

void ReadParameters(int argc, char* argv[], struct PARAM& parameters);

void ReadCircuit(std::string input, std::vector<struct GATE> &VectorGates, std::vector<int> &OUTPUTS, std::vector<int>& INPUTS);

void ReadInputsAndWrite(std::string input2, std::string output, std::vector<struct GATE>& VectorGates, std::vector<int> OUTPUTS, std::vector<int> INPUTS);

enum class GateType TranslateStringToEnum(std::string original);

void EvaluateOut(int nodeID, std::vector<struct GATE> &VectorGates);


```

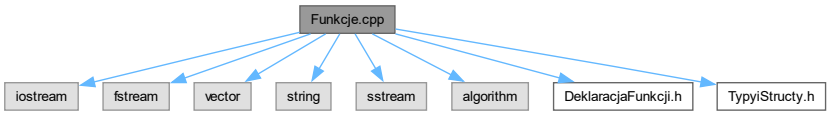
4.3 Dokumentacja pliku Funkcje.cpp

Funkcje potrzebne do programu.

```
#include <iostream>
#include <fstream>
#include <vector>
```

```
#include <string>
#include <sstream>
#include <algorithm>
#include "DeklaracjaFunkcji.h"
#include "TypyiStructy.h"
```

Wykres zależności załączania dla Funkcje.cpp:



Funkcje

- void CheckParameters (int argc, char ∗argv[ ])

  Funkcja sprawdzajaca czy zgadza sie liczba podanych argumentow.
- void ReadParameters (int argc, char ∗argv[ ], PARAM &parameters)

  Funkcja czytajaca argumenty i podajaca je do struktury z parametrami.
- void ReadCircuit (std::string input, std::vector< struct GATE > &VectorGates, std::vector< int > &OUTPUTS, std::vector< int > &INPUTS)

  Funkcja czytajaca uklad z pliku i budujaca go.
- void ReadInputsAndWrite (std::string input2, std::string output, std::vector< struct GATE > &VectorGates, std::vector< int > OUTPUTS, std::vector< int > INPUTS)

  Funkcja czytajaca wartosci na wejsciach z pliku i wprowadzajaca wartosci wejsc do ukladu oraz wprowadzajaca wartosci do kazdej bramki i wypisujaca wartosci wyjsc.
- enum GateType TranslateStringToEnum (std::string original)

  Funkcja "tlumaczaca" stringi z pliku na enum.
- void EvaluateOut (int nodeID, std::vector< struct GATE > &VectorGates)

  Funkcja wyliczajaca wartosc wyjscia z wartosci znajdujacej sie na wejsciu bramki.

4.3.1 Opis szczegółowy

Funkcje potrzebne do programu.

Autor

Igor Szyk

Wersja

0.1

Data

2024-01-28

4.3.2 Dokumentacja funkcji

4.3.2.1 CheckParameters()

```
void CheckParameters (
            int argc,
            char * argv[] )
```

Funkcja sprawdzajaca czy zgadza sie liczba podanych argumentow.

Parametry

argc	parametr dla ilosci argumentow
argv[]	parametr tablicy dla nazw plikow uzywanych w programie

Oto graf wywoływań tej funkcji:

4.3.2.2 EvaluateOut()

```
void EvaluateOut (
            int nodeID,
            std::vector< struct GATE > & VectorGates )
```

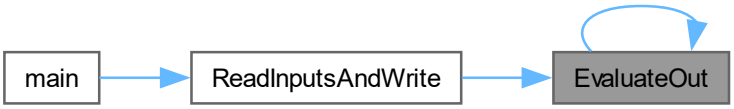
Funkcja wyliczajaca wartosc wyjscia z wartosci znajdujacej sie na wejsciu bramki.

Parametry

nodeID	numer wezla z ktorego aktualnie bedzie wyznaczana wartosc na wyjsciu
VectorGates	wektor zawierajacy uklad, na ktorym bedzie sie operowac

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:



4.3.2.3 ReadCircuit()

```
void ReadCircuit (
            std::string input,
            std::vector< struct GATE > & VectorGates,
            std::vector< int > & OUTPUTS,
            std::vector< int > & INPUTS )
```

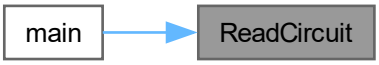
Funkcja czytajaca uklad z pliku i budujaca go.

Parametry

input	plik z ktorego czyta sie budowe ukladu
VectorGates	wektor do ktorego wpisuje sie wyczytany uklad z pliku
OUTPUTS	wektor do ktorego wypisuje sie numery bramek wyjscia
INPUTS	wektor do ktorego wypisuje sie numery bramek wejscia

Oto graf wywołań dla tej funkcji:

Oto graf wywoływań tej funkcji:



4.3.2.4 ReadInputsAndWrite()

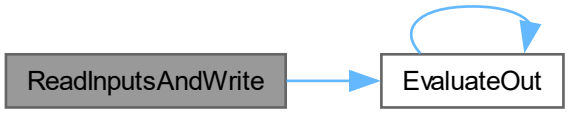
```
void ReadInputsAndWrite (
            std::string input2,
            std::string output,
            std::vector< struct GATE > & VectorGates,
            std::vector< int > OUTPUTS,
            std::vector< int > INPUTS )
```

Funkcja czytajaca wartosci na wejsciach z pliku i wprowadzajaca wartosci wejsc do ukladu oraz wprowadzajaca wartosci do kazdej bramki i wypisujaca wartosci wyjsc.

Parametry

input2	plik z ktorego czyta sie wartosci na wejsciach
output	plik do ktorego wypisuje sie wartosci wejsc i wyjsc
VectorGates	wektor na ktorym czyta sie wartosci wyjsc potrzebnych bramek
OUTPUTS	wektor numerow wyjsc do ktorych szuka sie wartosci
INPUTS	wektor numerow wejsc, ktore wykorzystuje sie do sprawdzenia czy zgadza sie ilosc wartosci z iloscia numerow wezlow

Oto graf wywołań dla tej funkcji:



Oto graf wywoływań tej funkcji:

4.3.2.5 ReadParameters()

```
void ReadParameters (
            int argc,
            char * argv[],
            struct PARAM & parameters )
```

Funkcja czytajaca argumenty i podajaca je do struktury z parametrami.

Parametry

argc	parametr dla ilosci argumentow
argv[]	parametr tablicy dla nazw plikow uzywanych w programie
parameters	parametr struktury do ktorej wstawia sie wyczytane argumenty

Oto graf wywoływań tej funkcji:

4.3.2.6 TranslateStringToEnum()

```
enum GateType TranslateStringToEnum (
            std::string original )  [strong]
```

Funkcja "tlumaczaca" stringi z pliku na enum.

Parametry

original	parametr zawierajacy oryginalny tekst w stringu

Zwraca

typ bramki w enum

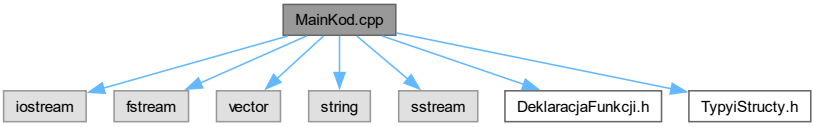
Oto graf wywoływań tej funkcji:

4.4 Dokumentacja pliku MainKod.cpp

Program tworzacy uklady cyfrowe i liczacy wartosci.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include "DeklaracjaFunkcji.h"
#include "TypyiStructy.h"
```

Wykres zależności załączania dla MainKod.cpp:



Funkcje

- int main (int argc, char *argv[])

  Glowna funkcja.

4.4.1 Opis szczegółowy

Program tworzacy uklady cyfrowe i liczacy wartosci.

Autor

Igor Szyk

Wersja

0.1

Data

2024-01-28

4.4.2 Dokumentacja funkcji

4.4.2.1 main()

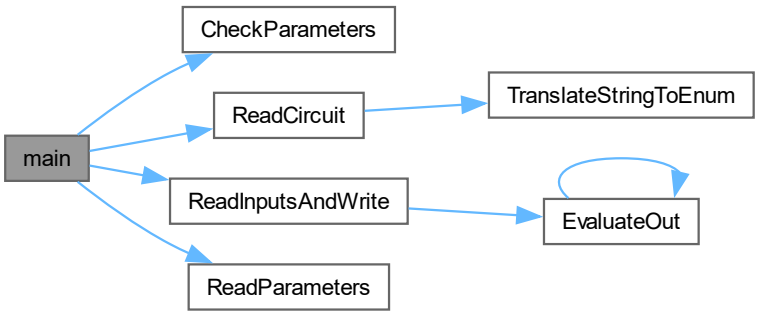
```
int main (
            int argc,
            char * argv[] )
```

Glowna funkcja.

Parametry

argc	podany argument
argv[]	nazwa pliku argumentu

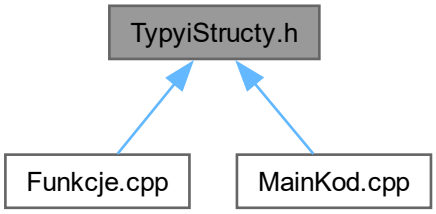
Oto graf wywołań dla tej funkcji:



4.5 Dokumentacja pliku TypyiStructy.h

Header zawierajacy struktury i typy wyliczeniowe.

Ten wykres pokazuje, które pliki bezpośrednio lub pośrednio załączają ten plik:



Komponenty

- struct GATE

  Struktura odzwierciedlajaca bramki.
- struct PARAM

  Struktura zawierajaca w sobie argumenty.

Wyliczenia

- enum class GateType {
  AND , **NAND** , **OR** , **NOR** ,
  XOR , **XNOR** , **NEG** , **IN** }

  Enum opisujacy typ bramki.
- enum class GateValue { **OFF** = 0 , **ON** = 1 , **UNIND** = -1 }

  Enum opisujacy wartosc na wyjsciu bramki.

4.5.1 Opis szczegółowy

Header zawierajacy struktury i typy wyliczeniowe.

Autor

Igor Szyk

Wersja

0.1

Data

2024-01-28

4.6 TypyiStructy.h

Idź do dokumentacji tego pliku.

```
00001
00010 #pragma once
00014 enum class GateType { AND, NAND, OR, NOR, XOR, XNOR, NEG, IN };
00015
00020 enum class GateValue { OFF = 0, ON = 1, UNIND = -1 };
00021
00026 struct GATE {
00027     GateType Name;
00028     std::vector<int> inputNodes;
00029     GateValue out = GateValue::UNIND;
00030 };
00031
00032
00037 struct PARAM {
00038     std::string u,
00039         i,
00040         o;
00041 };
00042
00043
```